

## Лабораторна робота № 8

### Сластнікової Анастасії

**Тема:** Розширені можливості Redis

**Мета:** Закріпити знання про роботу з Redis та ознайомитися з розширеним функціоналом — транзакціями, скриптами на Lua, публікацією/підпискою (Pub/Sub) та потоками Redis Streams.

```
127.0.0.1:6379> SET balance 100
OK
127.0.0.1:6379> SET purchases 0
OK
127.0.0.1:6379>
```

Створила початковий баланс

```
127.0.0.1:6379> WATCH balance
OK
127.0.0.1:6379> |
```

Увімкнула спостереження за ключем balance

```
127.0.0.1:6379> MULTI
OK
```

Відкрила транзакцію

```
127.0.0.1:6379(TX)> DECRBY balance 10
QUEUED
127.0.0.1:6379(TX)> INCR purchases
QUEUED
```

Додада команди до транзакції

```
127.0.0.1:6379(TX)> EXEC
1) (integer) 90
2) (integer) 1
127.0.0.1:6379> |
```

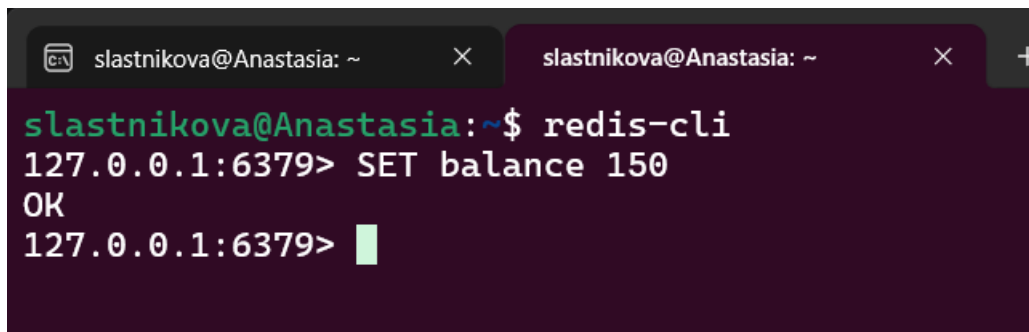
Виконана транзакція

```
127.0.0.1:6379> MULTI
OK
127.0.0.1:6379(TX)> SET user:1 "John"
QUEUED
127.0.0.1:6379(TX)> SET user:2 "Mary"
QUEUED
127.0.0.1:6379(TX)> SADD active:users "user:1" "user:2"
QUEUED
127.0.0.1:6379(TX)> EXEC
1) OK
2) OK
3) (integer) 2
127.0.0.1:6379> |
```

Проста транзакція, що додає кількість ключів

```
127.0.0.1:6379> WATCH balance
OK
127.0.0.1:6379> MULTI
OK
127.0.0.1:6379(TX)> DECRBY balance 20
QUEUED
127.0.0.1:6379(TX)> |
```

Встановила спостереження та почала транзакцію



```
slastnikova@Anastasia: ~
slastnikova@Anastasia: ~$ redis-cli
127.0.0.1:6379> SET balance 150
OK
127.0.0.1:6379> |
```

Змінила баланс в обхід транзакції

```
127.0.0.1:6379(TX)> EXEC
(nil)
127.0.0.1:6379> |
```

Транзакція повністю скасована, оскільки значення balance було змінено іншим клієнтом після виклику WATCH

```
127.0.0.1:6379> EVAL "if redis.call('exists', KEYS[1]) == 0 then return redis.call('set', KEYS[1], ARGV[1]) else return 'exists' end" 1 testkey "my_value"
OK
127.0.0.1:6379> |
```

Виконала команду EVAL (перевірка існування ключа та його створення, якщо він відсутній)

```
127.0.0.1:6379> SUBSCRIBE news
Reading messages... (press Ctrl-C to quit)
1) "subscribe"
2) "news"
3) (integer) 1
```

Запустила підписку на канал новин

```
127.0.0.1:6379> PUBLISH news "Redis is awesome!"
(integer) 1
127.0.0.1:6379> █
```

У Терміналі 2 можна побачити (integer) 1 (кількість активних підписників)

```
1) "message"
2) "news"
3) "Redis is awesome!"
|
```

Результат у терміналі 1

```
127.0.0.1:6379> XADD mystream * sensor-id 1234 temperature 19.8
"1778941729313-0"
127.0.0.1:6379> XADD mystream * sensor-id 1234 temperature 20.3
"1778941741181-0"
127.0.0.1:6379> |
```

Додавання подій у потік

```
127.0.0.1:6379> XRANGE mystream - +
1) 1) "1778941729313-0"
   2) 1) "sensor-id"
      2) "1234"
      3) "temperature"
      4) "19.8"
2) 1) "1778941741181-0"
   2) 1) "sensor-id"
      2) "1234"
      3) "temperature"
      4) "20.3"
127.0.0.1:6379> |
```

Читання подій з потоку. Діапазон від найменшого до найбільшого ID

```

127.0.0.1:6379> XREAD COUNT 2 STREAMS mystream 0
1) 1) "mystream"
   2) 1) 1) "1778941729313-0"
        2) 1) "sensor-id"
           2) "1234"
           3) "temperature"
           4) "19.8"
   2) 1) "1778941741181-0"
        2) 1) "sensor-id"
           2) "1234"
           3) "temperature"
           4) "20.3"
127.0.0.1:6379> |

```

Зчитування нових повідомлень за допомогою XREAD

- a. Визначте, які з нових механізмів Redis можуть бути корисними у великих розподілених системах.

У великих розподілених системах найкориснішими є механізми Redis Streams та Redis Cluster, які дозволяють будувати масштабовану архітектуру для обробки величезних потоків даних. Стріми ідеально підходять для логування подій та асинхронної взаємодії між мікросервісами, оскільки підтримують групи споживачів і гарантують доставку повідомлень, а кластеризація забезпечує автоматичне шардування даних між вузлами та високу відмовостійкість. Також круто себе показують механізми Client-side caching для мінімізації мережових затримок і Probabilistic data structures, які дозволяють миттєво перевіряти наявність елемента у гігантських базах, не витрачаючи зайву оперативну пам'ять. Ці інструменти роблять систему не просто швидкою, а стійкою до пікових навантажень і легкою в горизонтальному масштабуванні.

- b. Які плюси і мінуси Redis як брокера повідомлень?

Як брокер повідомлень Redis виділяється насамперед своєю неймовірною швидкістю, оскільки всі операції проходять в оперативній пам'яті, що забезпечує мінімальні затримки при передачі даних. Його величезний плюс у гнучкості: можна використовувати простий механізм Pub/Sub для миттєвих повідомлень або Redis Streams для складних сценаріїв із підтвердженням доставки та зберіганням історії подій. Проте є і суттєві мінуси — через зберігання даних в RAM ми сильно обмежені в обсягах черг, а у разі раптового збою живлення є ризик втратити частину повідомлень, які ще не встигли скинутися на диск. Крім того, у порівнянні з тим же RabbitMQ або Kafka, у Redis менше вбудованих інструментів для складного

маршрутизування та тривалого зберігання величезних архівів повідомлень, тому для критично важливих фінансових транзакцій його зазвичай використовують з обережністю.

Висновок: У межах лабораторної роботи було опановано просунуті можливості NoSQL-СУБД Redis. Під час практичних випробувань акцент зроблено на забезпеченні цілісності даних в умовах конкурентного доступу за допомогою транзакцій та механізму WATCH. Впровадження Lua-скриптів дозволило реалізувати атомарну обробку логіки безпосередньо на сервері. Також було досліджено архітектурні підходи до обміну повідомленнями через моделі Pub/Sub та Redis Streams. Програмна реалізація клієнта на Python підтвердила ефективність використання Redis як ключового компонента для створення високопродуктивних розподілених сервісів.